

Week 12 – Basic Storage, Partitions and LVM

Topics covered

- RH134 Chapter 7: Manage Basic Storage
- RH134 Chapter 8: Manage Storage Stack
- Identifying disks and partitions
- Creating partitions with **fdisk**
- Formatting partitions
- Mounting and unmounting filesystems
- Configuring **/etc/fstab**
- Creating and activating swap
- Physical volumes, volume groups and logical volumes
- Extending LVM storage
- Practical partitioning and LVM exercises

The Week 12 practical uses additional disks such as **/dev/sdb** and **/dev/sdc**. Do not run the partitioning commands against **/dev/sda**, because **/dev/sda** is normally the system disk containing Linux.

1. Understanding disks and partitions

Linux represents storage devices as files under:

/dev

Common SATA or virtual disk names are:

1. **/dev/sda**
2. **/dev/sdb**

/dev/sdc

Partitions are identified by numbers:

3. **/dev/sdb1**
4. **/dev/sdb2**

/dev/sdb3

Meaning:

Device	Meaning
--------	---------

/dev / s d a	First disk
--------------------------	------------

/dev / s d b	Second disk
--------------------------	-------------

/dev / s d b 1	First partition on the second disk
-------------------------------	------------------------------------

/dev / s d b 5	First logical partition on the second disk
-------------------------------	--

Before changing storage, identify the disks:

lsblk

More detailed filesystem information:

lsblk -f

List partition tables:

sudo fdisk -l

Check filesystem usage:

df -h

Do not assume that the additional disk is always `/dev/sdb`. Confirm with `lsblk`.

2. Primary, extended and logical partitions

On a traditional MBR partition table, a disk can have a maximum of four primary partition entries.

An extended partition acts as a container for logical partitions.

Example structure:

5. `/dev/sdb1` Primary
6. `/dev/sdb2` Primary
7. `/dev/sdb3` Primary
8. `/dev/sdb4` Extended
9. `/dev/sdb5` Logical
10. `/dev/sdb6` Logical

`/dev/sdb7` Logical

The extended partition itself normally does not contain a filesystem. Its logical partitions hold the usable storage.

In the Week 12 exercise:

- `/dev/sdb1` is a primary Linux partition.
 - `/dev/sdb2` is a primary Linux partition.
 - `/dev/sdb3` is a primary swap partition.
 - `/dev/sdb4` uses the remaining space as an extended partition.
 - `/dev/sdb5`, `/dev/sdb6` and `/dev/sdb7` are logical LVM partitions.
-

3. Viewing a partition table with `fdisk`

List all disks:

```
sudo fdisk -l
```

List a specific disk:

```
sudo fdisk -l /dev/sdb
```

Important fields include:

Field	Meaning
Device	Partition name
Start	Starting sector
End	Ending sector
Sectors or Blocks	Partition size
Id	Partition type identifier
Type or System	Human-readable partition type

The slides use examples such as:

- Type 83 for a Linux filesystem partition
 - Type 82 for Linux swap
 - Type 5 for an extended partition
-

4. Entering **fdisk** interactive mode

To modify **/dev/sdb**:

```
sudo fdisk /dev/sdb
```

Inside **fdisk**, common commands are:

K	Purpose
m	Display help
p	Print the current partition table
n	Create a new partition
d	Delete a partition
t	Change a partition type
l	List known partition types
w	Write changes and exit
q	Quit without saving

Before changing anything, print the table:

p

Changes remain in memory until **w** is entered. If a mistake is made before writing, use:

q

to exit without saving.

5. Creating a primary partition

Example: create a 400 MiB primary partition on **/dev/sdb**.

Start:

```
sudo fdisk /dev/sdb
```

Inside **fdisk**:

```
11. n
12. p
13. 1
14. Enter
```

```
+400M
```

Meaning:

- **n** creates a new partition.
- **p** selects primary.
- **1** assigns partition number 1.
- **Enter** accepts the suggested first sector.
- **+400M** sets the size to 400 MiB.

Print the proposed table:

p

Do not write until all partitions have been checked.

6. Creating an extended partition

After creating the required primary partitions, create an extended partition using the remaining disk space.

Inside **fdisk**:

- 15. n
- 16. e
- 17. 4
- 18. Enter

Enter

Meaning:

- **e** selects extended.
- **4** creates **/dev/sdb4**.
- The first **Enter** accepts the default starting sector.
- The second **Enter** uses the remaining disk space.

The extended partition becomes a container for logical partitions.

7. Creating logical partitions

After the extended partition exists, create logical partitions:

- 19. n
- 20. l
- 21. Enter

+200M

This normally creates:

/dev/sdb5

Create another:

- 22. n
- 23. l
- 24. Enter

+300M

This normally creates:

/dev/sdb6

Create a final logical partition using the remaining space:

25. n

26. l

27. Enter

Enter

This normally creates:

/dev/sdb7

8. Changing partition types

A newly created partition may default to the Linux filesystem type.

To change a partition type:

t

Select the partition number, then enter the required type.

Typical types in this lab are:

Purpose	Traditional code
Linux filessystem	83
Linux swap	82
Linux LVM	8e

Example: change partition 3 to swap:

28. t

29. 3

82

Change partition 5 to LVM:

30. t

31. 5

8e

Repeat for partitions 6 and 7.

On newer versions of **fdisk**, the interface may ask you to choose the type by name or from a numbered list. Select:

Linux swap

or:

Linux LVM

as required.

9. Writing the partition table

After checking with:

p

save the changes:

w

The partition table is then written to disk.

Ask the kernel to reread it:

`sudo partprobe /dev/sdb`

Verify:

32. lsblk

`sudo fdisk -l /dev/sdb`

If the kernel cannot reread the table because the device is busy, a reboot may be required.

10. Creating a filesystem

A partition cannot normally store regular files until a filesystem has been created.

The slides use **mkfs** to format a partition as **ext4**.

Format **/dev/sdb1**:

```
sudo mkfs.ext4 /dev/sdb1
```

Alternative syntax:

```
sudo mkfs -t ext4 /dev/sdb1
```

Format **/dev/sdb2**:

```
sudo mkfs.ext4 /dev/sdb2
```

Verify:

```
lsblk -f
```

or:

```
sudo blkid /dev/sdb1 /dev/sdb2
```

Formatting destroys existing data on the selected partition.

11. Understanding mount points

A filesystem must be attached to the Linux directory tree before its files can be accessed.

Create mount points:

```
33. sudo mkdir -p /mnt/sdb1
```

```
sudo mkdir -p /mnt/sdb2
```

Mount the filesystems:

34. `sudo mount /dev/sdb1 /mnt/sdb1`

`sudo mount /dev/sdb2 /mnt/sdb2`

Verify:

35. `findmnt /mnt/sdb1`

`findmnt /mnt/sdb2`

or:

`df -h`

Test:

36. `sudo touch /mnt/sdb1/testfile`

`ls -l /mnt/sdb1`

12. Unmounting a filesystem

Unmount by device:

`sudo umount /dev/sdb1`

or by mount point:

`sudo umount /mnt/sdb1`

Verify:

`findmnt /mnt/sdb1`

If the filesystem is busy, check which processes are using it:

`sudo lsof +f -- /mnt/sdb1`

or:

`sudo fuser -vm /mnt/sdb1`

A common mistake is typing **unmount**. The correct command is:

`umount`

13. Temporary versus persistent mounting

This command mounts a filesystem immediately:

```
sudo mount /dev/sdb1 /mnt/sdb1
```

However, the mount does not automatically return after reboot.

Persistent mounts are configured in:

```
/etc/fstab
```

The Week 12 slides recommend using the filesystem UUID rather than the device name because names such as `/dev/sdb1` may change if disk detection order changes.

14. Finding the UUID

Use:

```
sudo blkid /dev/sdb1
```

Example:

```
/dev/sdb1: UUID="abcd-1234" TYPE="ext4"
```

Another method:

```
lsblk -f
```

The UUID uniquely identifies the filesystem.

15. Understanding `/etc/fstab`

Each `/etc/fstab` entry normally contains six fields:

Device	Mount-point	Filesystem	Options	Dump	Check
--------	-------------	------------	---------	------	-------

Example:

UUID=abcd-1234	/mnt/sdb1	ext4	defaults	0	0
----------------	-----------	------	----------	---	---

Meaning:

Field	Value	Purpose
Device	UUID=abcd-1234	Filesystem identifier
Mount point	/mnt/sdb1	Directory where it is mounted
Filesystem	ext4	Filesystem type
Options	defaults	Standard mount options
Dump	0	Do not use the dump backup field
Check	0	No automatic filesystem check ordering for this lab

The slides state that non-root devices can use 0 0 for the final fields for simplicity.

16. Adding a persistent mount

First back up the file:

```
sudo cp /etc/fstab /etc/fstab.backup
```

Retrieve UUIDs:

```
sudo blkid /dev/sdb1 /dev/sdb2
```

Edit:

```
sudo vi /etc/fstab
```

Add entries similar to:

```
37. UUID=<UUID-of-sdb1> /mnt/sdb1 ext4 defaults 0 0
```

```
UUID=<UUID-of-sdb2> /mnt/sdb2 ext4 defaults 0 0
```

Test without rebooting:

```
sudo mount -a
```

Verify:

```
38. findmnt /mnt/sdb1
```

```
findmnt /mnt/sdb2
```

If **mount -a** returns no error, the entries were generally accepted.

Never reboot immediately after editing **/etc/fstab** without testing **mount -a**, because an invalid entry may interrupt the boot process.

17. Creating swap storage

The exercise uses **/dev/sdb3** as a 100 MiB swap partition.

Create the swap signature:

```
sudo mkswap /dev/sdb3
```

Activate it:

```
sudo swapon /dev/sdb3
```

Verify:

```
swapon --show
```

or:

```
free -h
```

Deactivate:

```
sudo swapoff /dev/sdb3
```

18. Making swap persistent

Find the UUID:

```
sudo blkid /dev/sdb3
```

Add to **/etc/fstab**:

```
UUID=<UUID-of-sdb3> none swap defaults 0 0
```

Activate all swap entries:

```
sudo swapon -a
```

Verify:

```
swapon --show
```

The mount-point field for swap is normally:

```
none
```

and the filesystem type is:

```
swap
```

LVM storage stack

19. Why LVM is used

Logical Volume Management adds a flexible layer between physical storage and filesystems.

The storage flow is:

39. Disk or partition

40. ↓

41. Physical Volume

42. ↓
43. **Volume Group**
44. ↓
45. **Logical Volume**
46. ↓
47. **Filesystem**
48. ↓

Mount point

LVM allows an administrator to:

- **Combine storage from several partitions or disks**
 - **Create multiple logical volumes from a shared storage pool**
 - **Extend logical volumes later**
 - **Manage space more flexibly than fixed partitions**
-

20. LVM terminology

T	Meaning
P	Physical Volume
V	Volume Group
L	Logical Volume
P	Physical Extent

L Logical Extent

A physical volume is an LVM-initialised disk or partition.

A volume group combines one or more physical volumes into a pool.

A logical volume is storage allocated from the volume group.

Logical volumes appear under paths such as:

`/dev/vg1/lv1`

They may also appear as:

`/dev/mapper/vg1-lv1`

21. Checking LVM storage

Useful commands:

49. `sudo pvs`

50. `sudo vgs`

`sudo lvs`

Detailed versions:

51. `sudo pvdisplay`

52. `sudo vgdisplay`

`sudo lvdisplay`

Check block devices:

`lsblk`

22. Creating physical volumes

The exercise uses:

53. `/dev/sdb5`
54. `/dev/sdb6`
55. `/dev/sdb7`

`/dev/sdc1`

Create physical volumes:

```
sudo pvcreate /dev/sdb5 /dev/sdb6 /dev/sdb7 /dev/sdc1
```

Verify:

```
sudo pvs
```

Expected result: all four devices appear as LVM physical volumes.

23. Creating volume groups

According to the final Week 12 LVM diagram:

- **vg1** is approximately 750 MiB.
- **vg2** is approximately 1.3 GiB.
- `/dev/sdb5`, `/dev/sdb6`, `/dev/sdb7` and `/dev/sdc1` are the physical volumes.
- **vg1** receives the approximately 200 MiB and 550 MiB physical volumes.
- **vg2** receives the approximately 300 MiB and 1 GiB physical volumes.

Create **vg1**:

```
sudo vgcreate vg1 /dev/sdb5 /dev/sdb7
```

Create **vg2**:

```
sudo vgcreate vg2 /dev/sdb6 /dev/sdc1
```

Verify:

56. `sudo vgs`
57. `sudo vgdisplay vg1`

```
sudo vgdisplay vg2
```

The exact displayed capacity may be slightly smaller than the stated partition sizes because LVM metadata and extent rounding consume some space.

24. Creating logical volumes by size

The diagram requires:

`/dev/vg1/lv1 = 400 MiB`

Create it:

```
sudo lvcreate -L 400M -n lv1 vg1
```

Meaning:

- `-L 400M` specifies the size.
- `-n lv1` specifies the logical-volume name.
- `vg1` identifies the source volume group.

Verify:

```
sudo lvs
```

25. Creating a logical volume using remaining space

The diagram shows that `lv2` receives approximately the remaining 350 MiB of `vg1`.

Create it using all remaining extents:

```
sudo lvcreate -l 100%FREE -n lv2 vg1
```

The lowercase `-l` specifies extents or a percentage rather than a direct byte size.

Verify:

```
58. sudo lvs
```

```
sudo vgs
```

`vg1` should now have little or no unallocated space.

26. Creating logical volumes by extents

The diagram states:

`/dev/vg2/lv3 = 300 extents`

Create it:

```
sudo lvcreate -l 300 -n lv3 vg2
```

It also shows:

`/dev/vg2/lv4 = approximately 25 extents`

Create it using the remaining extents:

```
sudo lvcreate -l 100%FREE -n lv4 vg2
```

If the exercise requires exactly 25 extents:

```
sudo lvcreate -l 25 -n lv4 vg2
```

The slide describes the final amount as approximately 25 extents, so **100%FREE** is the safer way to consume whatever remains after extent rounding.

Verify:

```
sudo lvs -o lv_name,vg_name,lv_size
```

27. Formatting logical volumes

A logical volume must still be formatted before it can store regular files.

Example:

```
59. sudo mkfs.ext4 /dev/vg1/lv1
```

```
60. sudo mkfs.ext4 /dev/vg1/lv2
```

```
61. sudo mkfs.ext4 /dev/vg2/lv3
```

```
sudo mkfs.ext4 /dev/vg2/lv4
```

Verify:

```
lsblk -f
```

28. Mounting logical volumes

Create mount points:

62. `sudo mkdir -p /mnt/lv1`

63. `sudo mkdir -p /mnt/lv2`

64. `sudo mkdir -p /mnt/lv3`

`sudo mkdir -p /mnt/lv4`

Mount:

65. `sudo mount /dev/vg1/lv1 /mnt/lv1`

66. `sudo mount /dev/vg1/lv2 /mnt/lv2`

67. `sudo mount /dev/vg2/lv3 /mnt/lv3`

`sudo mount /dev/vg2/lv4 /mnt/lv4`

Verify:

68. `findmnt | grep '/mnt/lv'`

`df -h`

The final LVM slide specifies the LV names and sizes, but it does not specify these mount-point names. `/mnt/lv1` to `/mnt/lv4` are therefore clear practice choices rather than fixed answers from the slide.

29. Persistent logical-volume mounting

Find UUIDs:

69. `sudo blkid /dev/vg1/lv1`

70. `sudo blkid /dev/vg1/lv2`

71. `sudo blkid /dev/vg2/lv3`

`sudo blkid /dev/vg2/lv4`

Add suitable entries to `/etc/fstab`:

72. `UUID=<lv1-uuid> /mnt/lv1 ext4 defaults 0 0`

73. `UUID=<lv2-uuid> /mnt/lv2 ext4 defaults 0 0`

74. `UUID=<lv3-uuid> /mnt/lv3 ext4 defaults 0 0`

`UUID=<lv4-uuid> /mnt/lv4 ext4 defaults 0 0`

Test:

```
sudo mount -a
```

Verify:

```
75. findmnt /mnt/lv1
```

```
76. findmnt /mnt/lv2
```

```
77. findmnt /mnt/lv3
```

```
findmnt /mnt/lv4
```

30. Extending a volume group

Suppose a new physical volume is available:

```
/dev/sdd1
```

Initialise it:

```
sudo pvcreate /dev/sdd1
```

Add it to **vg1**:

```
sudo vgextend vg1 /dev/sdd1
```

Verify:

```
78. sudo pvs
```

```
sudo vgs
```

The volume group now has more free space.

31. Extending a logical volume

Extend **lv1** by 200 MiB:

```
sudo lvextend -L +200M /dev/vg1/lv1
```

Verify:

```
sudo lvs
```

This increases the logical volume, but the filesystem may still need to be expanded separately.

For an ext4 filesystem:

```
sudo resize2fs /dev/vg1/lv1
```

A combined command may be used:

```
sudo lvextend -r -L +200M /dev/vg1/lv1
```

The **-r** option resizes the filesystem together with the logical volume.

32. Removing LVM storage

Unmount first:

```
sudo umount /mnt/lv1
```

Remove the logical volume:

```
sudo lvremove /dev/vg1/lv1
```

Remove the volume group after all its logical volumes have been removed:

```
sudo vgremove vg1
```

Remove the LVM label from physical volumes:

```
sudo pvremove /dev/sdb5 /dev/sdb7
```

Removal destroys data. Verify device names before confirming.

79.